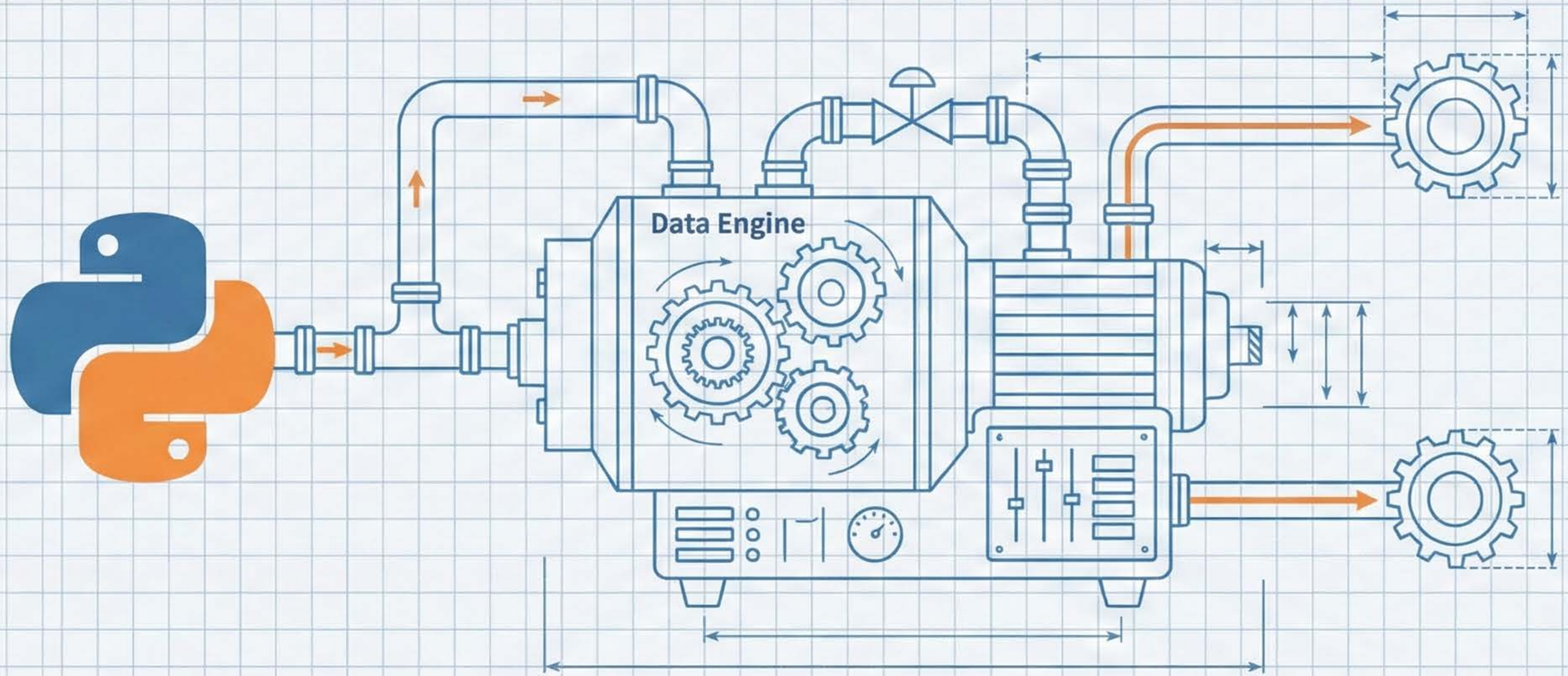


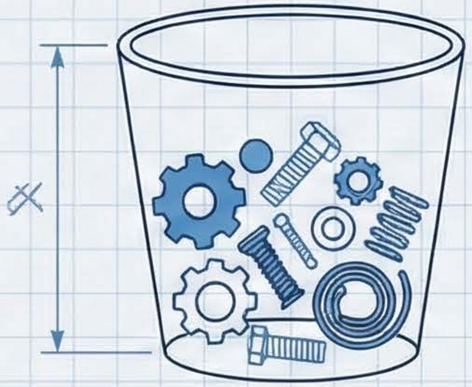
Unit 03 NumPy 數值計算基礎

Section 1: NumPy 基本概念 — 高效能運算的引擎

為什麼需要 NumPy?

從手動處理升級為工業級自動化產線

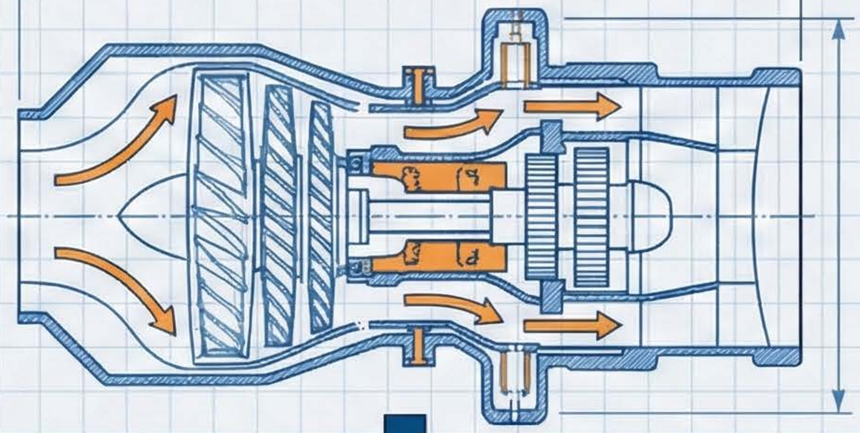
Python List



靈活但緩慢
(Flexible but Slow)

10x - 100x 效能提升
(Speedup)

NumPy Array

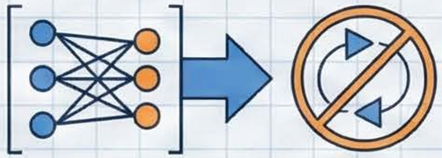


專為速度設計
(Designed for Speed)

高效能 (High Performance)

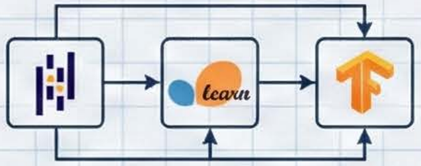
C 語言核心實作，大幅提升運算速度。

向量化 (Vectorization)



像像作單一變數一樣操作整組數據，無需撰寫迴圈。

生態系基石 (Foundation)



Pandas、Scikit-learn、TensorFlow 的基礎建設。

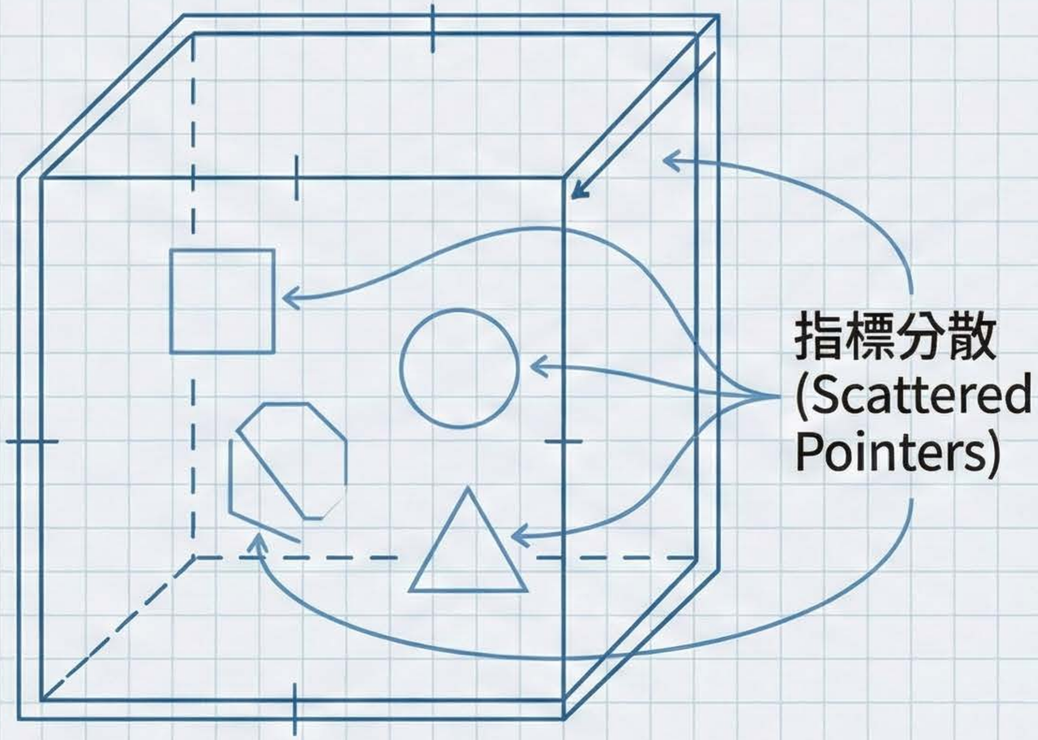
化工應用 (ChemE Apps)

製程數據處理、數值模擬、反應動力學計算。

什麼是 NumPy 陣列 (ndarray)？

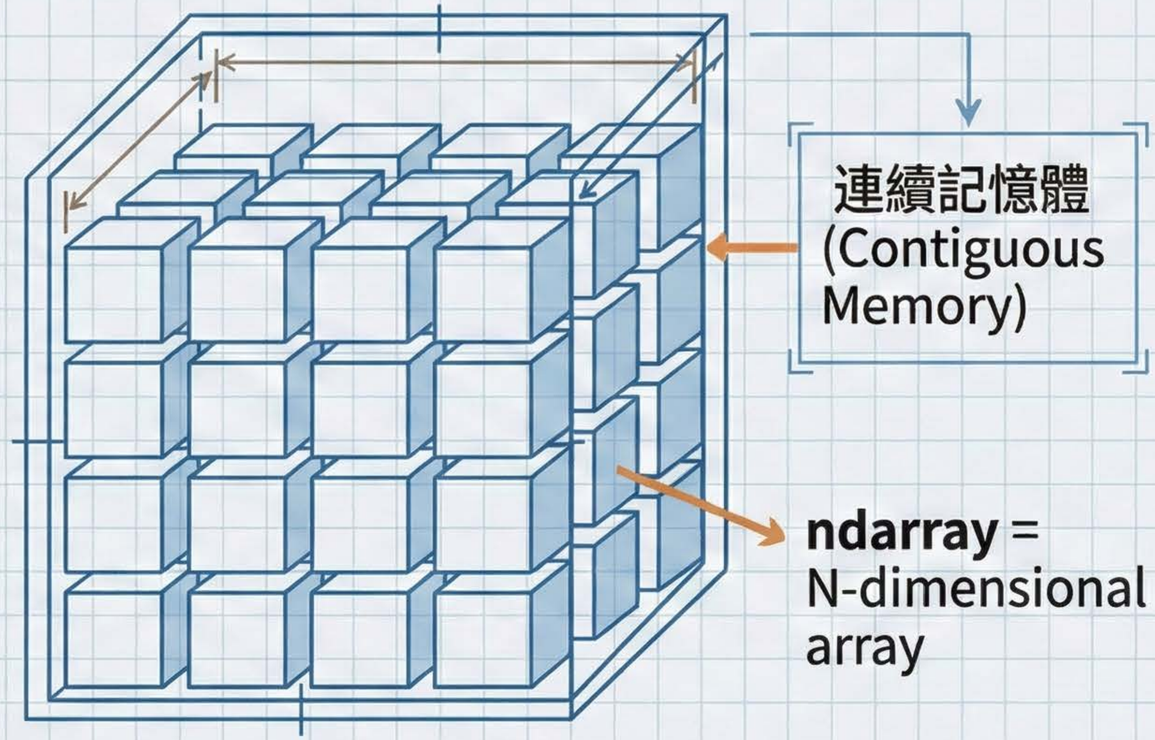
多維度的同質資料容器

Python List (異質性 / Heterogeneous)



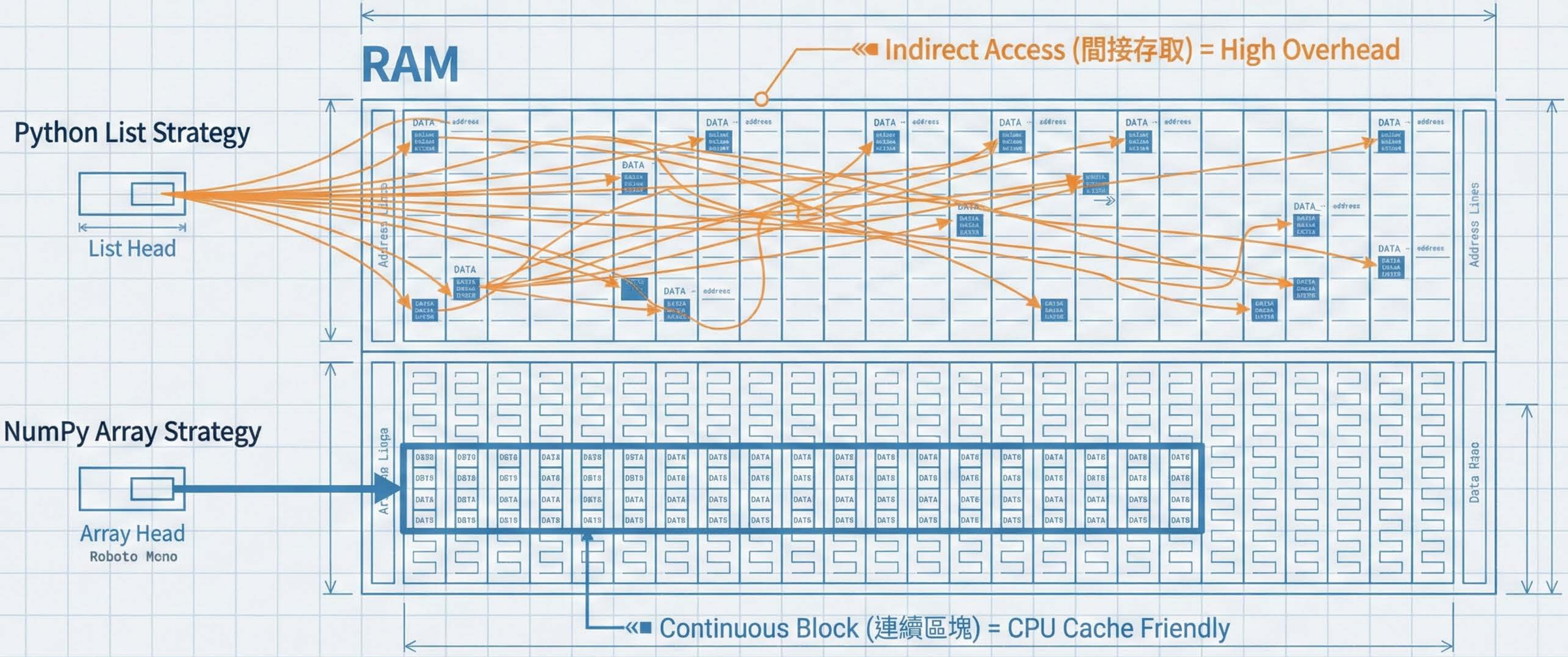
可包含不同型態 (Mixed Types)

NumPy Array (同質性 / Homogeneous)



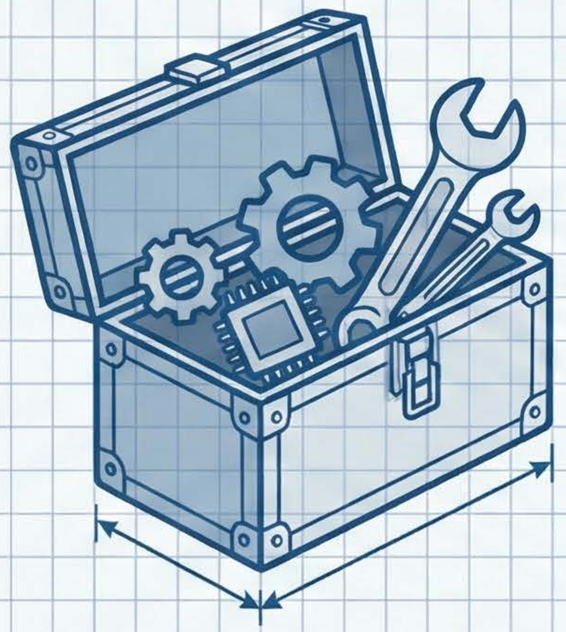
所有元素必須相同型態 (Strictly Single Type)

效能的秘密：記憶體配置 (Memory Layout)



核心權衡 (Trade-off)：NumPy 犧牲了靈活性（強制單一型態），換取了極致的運算速度與記憶體效率。

準備您的工具箱 (Installation & Import)



1. 在終端機 (Terminal) 執行：

```
pip install numpy
```

2. 在 Python 程式中匯入：

```
import numpy as np
```

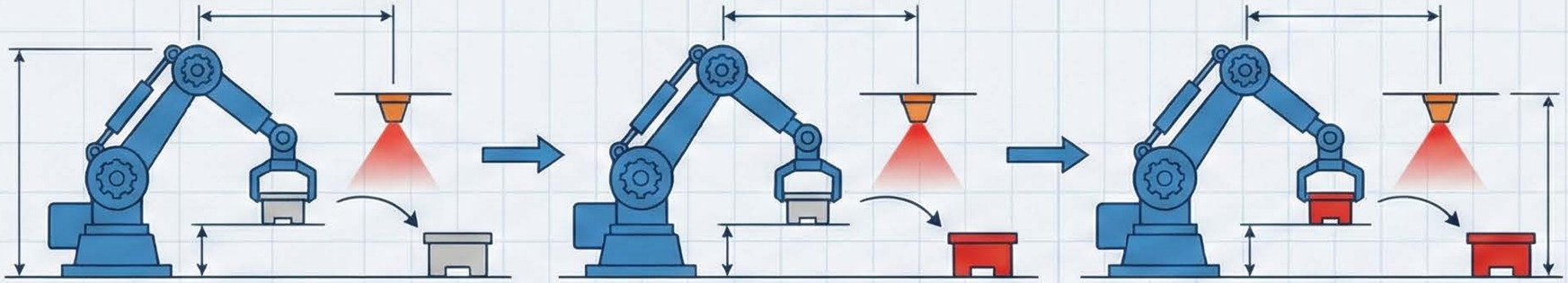
重要慣例 (Convention)：
`np` 是業界通用的標準縮寫 (Standard Alias)。為了與其他工程師協作，請務必遵守此慣例。

NotebookLM

核心概念：向量化運算 (Vectorization)

對陣列執行運算時，會自動應用於每一個元素，無需撰寫迴圈 (No Loops)。

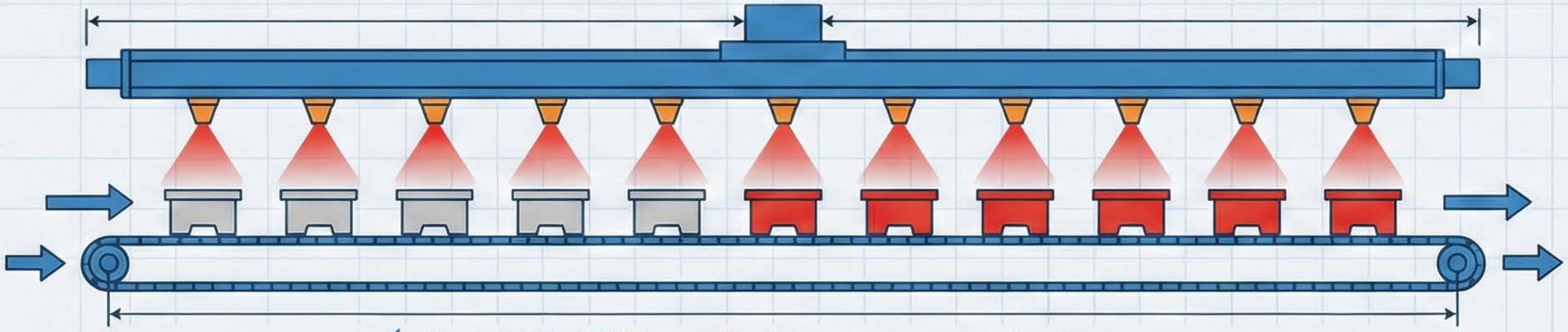
Explicit Loop (明確迴圈)



Step 1: Pick & Paint　　Step 2: Pick & Paint　　Step 3: Pick & Paint

逐一處理 (Item-by-Item) - Slow

Vectorization (向量化)



批次平行處理 (Parallel Batch Processing) - Fast

$y = f(x)$ applies to vector $\mathbf{x}$ at once.

實戰範例：溫度轉換 (Temperature Conversion)

將攝氏 (Celsius) 轉換為克氏 (Kelvin)：K = C + 273.15

Python List (The Old Way)

```
celsius = [25.5, 30.1, 100.0]
kelvin = []
for c in celsius:
    kelvin.append(c + 273.15)
```

Verbose & Slower (冗長且較慢)

NumPy Array (The NumPy Way)

```
celsius_arr = np.array([25.5, 30.1, 100.0])
kelvin_arr = celsius_arr + 273.15
```

Concise & Fast (簡潔且高效)

純量廣播 (Scalar Broadcast):
自動將 `273.15` 加到每個元素。

NotebookLM

本節重點回顧 (Section Recap)

QUALITY CONTROL CHECKLIST

- ☑ **ndarray**: 同質性、連續記憶體、固定大小 (Homogeneous, Contiguous, Fixed Size).
- ☑ **Efficiency**: C 語言核心，比 List 快 **10-100 倍** (10-100x Faster).
- ☑ **Vectorization**: 消除迴圈，程式碼更簡潔 (Eliminates Loops).
- ☑ **Convention**: import numpy as **np**.

Next Section:
建立 NumPy 陣列
(Creating Arrays)

ACTIVE WORKFLOW

“掌握了 NumPy，您就掌握了 Python 科學運算的基石。”

Notebook